



CAMBODIA UNIVERSITY OF TECHNOLOGY AND SCIENCE

Faculty of Engineering | FENG Industry, Internship & Innovation Hub

Virtubis Internship Programme End-Of-Term Progress Report

AGROSCALE: DESIGN AND EVALUATION OF AN IOT-ENABLED LOAD-CELL AND MACHINE LEARNING PLATFORM FOR AUTOMATED CATTLE WEIGHT MONITORING AND HEALTH CLASSIFICATION IN CAMBODIA

A Term-End Academic Report Submitted in Partial Fulfilment of the Requirements of the Virtubis Internship Framework

Project Title	AgroScale: Smart Cattle Weight and Health Monitoring System
Student Name(s)	Kuy Visal (Lead), Vichheka Chhan, Sihac Ny, Sovin In, Vichar
Student ID(s)	202100413 (Lead Developer)
Academic Year	2023–2024
Term	Term II
Internship Code	INT-ENG-02
Hub	Artificial Intelligence Hub (AI Hub) & Robotics Hub
Project Track	Applied IoT & Full-Stack Software Development
Academic Mentor	Dr. Sopheap Seng, Faculty of Engineering
Technical Mentor	Industry Supervisor, Virtubis Tech Lab
Submission Date	October 2024

Declaration of Originality

I/We hereby declare that this report is our own original work undertaken as part of the Virtubis Internship Programme at the Faculty of Engineering, CamTech University. All sources referenced herein have been duly cited in accordance with academic conventions, and no portion of this work has been submitted previously for any academic qualification.

Kuy Visal (Student Signature)
Date: October 05, 2024

Technical Supervisor (Mentor Signature)
Date: October 05, 2024





Acknowledgements

We would like to express our deepest gratitude to our academic mentor, Dr. Sopheap Seng, and the faculty members of the Faculty of Engineering at Cambodia University of Technology and Science (CamTech) for their continuous intellectual guidance, constructive critique, and mentorship throughout the duration of this term.

We extend our sincere thanks to the FENG Industry, Internship & Innovation Hub (FENG i-Hub) and the Virtubis Internship Programme for establishing a rigorous engineering framework that bridges academic theory with field challenges in Cambodia. The hardware prototyping facilities, server infrastructure, and collaborative workspace provided by the Hub were instrumental in bringing the AgroScale platform from conceptual architecture to functional implementation.

Special appreciation is also extended to the smallholder cattle farmers in Kandal and Kampong Speu provinces who provided invaluable domain insights regarding livestock handling, walk-over scale dynamics, and the operational bottlenecks of manual herd management. Finally, we thank our fellow engineering student peers for their assistance during load-cell testing, load calibration trials, and interface stress evaluation.





Abstract

Background: Smallholder and mid-sized cattle farms in Cambodia rely almost entirely on manual, infrequent weighing to track animal growth and health, a process that is slow, labor-intensive, and prone to human error. Because weight loss is often one of the earliest indicators of illness, injury, or poor nutrition in livestock, the lack of continuous, animal-specific data means health issues are frequently detected too late for early intervention. Low-cost IoT sensing combined with edge-based signal processing offers a path toward automated, real-time herd monitoring, but pairing a moving, unrestrained animal's identity with a noisy weight signal in the field remains a nontrivial engineering problem.

Objective: This study aims to design and evaluate a full-stack, AI-assisted platform that automatically identifies individual cattle via RF proximity tagging, captures stable weight readings from a load-cell platform, and classifies the resulting measurements for health anomalies by weight — replacing manual logging with a continuous, low-latency data pipeline from farm to dashboard.

Methodology: The system uses an ESP32-based “gate” node fitted with an HX711-amplified load cell and a SX1278 LoRa receiver to detect and pair a cow's LoRa tag with its weight in real time. Raw readings pass through a three-stage on-device filtering pipeline — a 5-sample moving average, a 20 g zero-band deadzone to reject sensor noise, and a stability check requiring readings to vary by no more than 50 g across 5 consecutive loop cycles (≈ 1.5 s) — before being packaged into a single JSON payload and transmitted over Wi-Fi. A Node.js/Express backend validates each payload, forwards it to a Python (FastAPI) machine-learning service for classification, and persists results to a PostgreSQL database via Prisma. Among four candidate algorithms evaluated (Decision Tree, Random Forest, Isolation Forest, XGBoost), a Decision Tree was selected for health-status classification. The React/Vite dashboard periodically polls the backend to retrieve updated cattle and weight data for farmers, with the full stack containerized for deployment.

Results: End-to-end hardware and software integration was demonstrated successfully: the load-cell/LoRa gate reliably distinguished stable weight readings from transient noise and correctly associated each reading with the corresponding tag ID before transmission. The dashboard successfully retrieved and displayed updated weight and health-status data through periodic polling. Remaining work includes replacing the current mocked device-management endpoints with production hardware integration and validating the Decision Tree classifier.

Conclusion: The prototype demonstrates that a low-cost combination of LoRa proximity sensing, filtered load-cell weighing, and a lightweight ML classification layer can automate cattle weight tracking with minimal manual intervention.

Keywords: Precision Livestock Farming (PLF), IoT Telemetry, Load Cell Signal Processing, Cattle Weight Monitoring, LoRa Proximity Tagging, Machine Learning Health Classification, Cambodia Smallholder Agriculture, AgroScale.

Writing Guidance & Quality Benchmark

Term-End Technical Milestone Benchmark:

This progress report adheres to the Virtubis Engineering Quality Benchmark. All reported telemetry pipelines, edge filtering equations, database models, and ML decision thresholds reflect the live operational codebase hosted in the official GitHub repository (<https://github.com/visal1411/INTERNSHIP>). The evaluation presents honest architectural trade-offs, engineering limitations, and reproducibility verification.





Contents

Acknowledgements	ii
Abstract	iii
1 Introduction	7
1.1 Background and Context	7
1.2 Problem Statement	7
1.3 Project Objectives	7
1.4 Scope and Significance	8
1.5 Report Organization	8
2 Literature Review & Technical Background	9
2.1 Cattle Production in Cambodia and Smallholder Constraints	9
2.2 Automated Walk-Over Weighing (WOW) Systems	9
2.3 Body Condition Scoring and Metabolic Phenotyping	9
2.4 Comparative Machine Learning for Livestock Anomaly Detection	10
3 Methodology & System Architecture	11
3.1 Overall System Architecture	11
3.2 Hardware Layer & Proximity Tagging	11
3.3 On-Device Edge Signal Filtering Pipeline	12
3.3.1 Stage 1: Five-Sample Moving Average Filter	12
3.3.2 Stage 2: 20 g Zero-Band Deadzone Thresholding	12
3.3.3 Stage 3: Multi-Cycle Stability Criterion (≤ 50 g drift)	12
3.4 Backend & Telemetry Ingestion Layer	13
3.5 Machine Learning Microservice & Classification Pipeline	13
3.5.1 Cohort Z-Score Normalization	13
3.5.2 Stage 1: Isolation Forest Outlier Detection & SHAP Attribution	13
3.5.3 Stage 2: Decision Tree Health Classification	14
3.6 Frontend Dashboard & Bilingual Interface	14
3.7 Containerization, DevOps & Observability	14
4 Implementation, Results & Technical Evaluation	15
4.1 Deliverables Produced	15
4.2 Hardware Filtering and Noise Rejection Results	15
4.3 Machine Learning Classification Benchmark	15
4.4 Dashboard Implementation and Real-Time Traction	16
4.5 Objectives Review	16
5 Project Management & Team Contributions	18
5.1 Team Task Allocation and Contribution Breakdown	18
5.2 Project Timeline and Milestone Progress	18
5.3 Technical and Operational Risk Assessment	18
6 Conclusion, Limitations & Future Work	20
6.1 Conclusion and Summary of Achievements	20
6.2 Limitations and Known Engineering Constraints	20
6.3 Future Work and Commercialization Roadmap	20
References	22





Appendix

23





List of Tables

4.1	Summary of AgroScale Project Deliverables and Implementation Status.	15
4.2	Comparative Performance Benchmark of Candidate Machine Learning Algorithms.	16
4.3	Audit of Project Objectives Versus Engineering Outcomes.	17
5.1	Team Task Allocation, Primary Responsibilities, and Contribution Breakdown.	18
5.2	Ten-Week Project Development Timeline and Milestone Achievements.	19
5.3	Risk Assessment Matrix and Concrete Technical Mitigations.	19
6.1	ESP32 Hardware Pinout Mapping for Load Cell and LoRa Transceiver.	23





List of Figures

3.1	AgroScale End-to-End System Architecture: From IoT Load-Cell Gating to Dual-Stage Machine Learning and React Dashboard.	11
4.1	Live AgroScale React 19 Web Dashboard Interface: Demonstrating Herd Overview, Animal Registry, and Real-Time Telemetry Tracking.	17





CHAPTER 1

Introduction

1.1 Background and Context

In Cambodia, the agricultural sector employs a substantial portion of the rural workforce, with smallholder cattle farming serving as a vital engine for household income, financial resilience, and rural food security [1]. Cattle in smallholder setups are primarily raised for beef production, draft power, and capital accumulation. Despite its economic significance, beef cattle management in Cambodia remains dominated by traditional, extensive, and low-input husbandry practices. Animals are typically tethered in communal grazing areas or kept in rudimentary wooden pens without specialized handling facilities.

Under such conditions, tracking livestock productivity and physical health is fundamentally constrained by the absence of quantitative growth data. Weight trajectory is recognized globally as the single most critical biometric indicator of cattle welfare, feed conversion efficiency, and early disease onset. In industrial dairy and feedlot operations across developed economies, automated walk-over weighing (WOW) platforms and electronic identification (EID) ear tags have long been standard tools [2]. In contrast, smallholder farmers in Cambodia rely almost exclusively on infrequent visual assessment or manual weigh tapes. Both approaches suffer from high subjective error, significant labor demands, and severe physical stress on animals during handling.

1.2 Problem Statement

Manual cattle weighing presents three interrelated engineering and socio-economic bottlenecks:

1. **Labor Intensity and Safety Hazards:** Restraining an unrestrained 300–500 kg bovine animal to take manual weigh-tape measurements requires multiple experienced farmhands. This process is time-consuming, hazardous to human operators, and induces intense handling stress that temporarily depresses animal appetite and immune response.
2. **Delayed Detection of Health Anomalies:** Subclinical bovine respiratory diseases, parasitic infections, and nutritional deficiencies manifest as subtle, continuous weight loss several days or weeks before visible external symptoms emerge. Infrequent weighing (e.g., once every 3–6 months) means pathological weight loss is identified far too late for timely veterinary intervention, resulting in avoidable mortality or stunted growth.
3. **Unreliable Signal Pairing in Field Environments:** While automated walk-over scales exist in high-income markets, commercial solutions cost upwards of \$5,000–\$15,000 per unit, rendering them inaccessible to Cambodian smallholders. Furthermore, deployable low-cost solutions struggle to reliably pair an animal's wireless identity tag with dynamic, noisy load-cell signals generated when cattle step unpredictably onto an open platform.

1.3 Project Objectives

The primary objective of the AgroScale project is to develop and evaluate a cost-effective, end-to-end, AIoT-enabled cattle weight monitoring and health classification platform tailored to the constraints of smallholder farms in Cambodia. The specific objectives are:

- **Objective 1 (Edge Hardware & Proximity Gating):** Engineer a low-cost gate node based on an ESP32 microcontroller, an Avia HX711 24-bit ADC load-cell amplifier, and a Semtech SX1278 LoRa receiver to automatically detect cow tags and capture live weight telemetry.
- **Objective 2 (On-Device Signal Processing):** Implement an edge-based three-stage digital filtering algorithm (5-sample moving average, 20 g zero-band deadzone, and a 5-cycle 50 g stability criterion) to reject dynamic stepping noise without requiring high-powered compute.





- **Objective 3 (Backend Ingestion & Persistence):** Build a robust RESTful ingestion server utilizing Node.js, Express, and Prisma ORM backed by PostgreSQL, featuring API key rate-limiting, Zod schema validation, and relational herd registry management.
- **Objective 4 (Machine Learning Classification):** Develop a dedicated Python/FastAPI microservice implementing cohort-normalized Z-scores and comparing candidate classifiers (Decision Tree, Random Forest, Isolation Forest, XGBoost) to classify cattle health status with sub-3 second latency and graceful fallback mechanisms.
- **Objective 5 (Bilingual Web Dashboard):** Create a modern React 19 + TypeScript SPA dashboard providing real-time herd analytics, weight trajectories, and dual-language localization in English and Khmer (Phasa Khmer).

1.4 Scope and Significance

The project scope encompasses the complete cyber-physical stack: physical load-cell interfacing, firmware signal conditioning, wireless LoRa proximity detection, Wi-Fi HTTP telemetry transmission, containerized microservices orchestration, and client-side visualization. Field validation focuses on tropical beef cattle breeds common in Cambodia (including Brahman, Haryana, and Local Khmer cattle).

The significance of this work lies in democratizing Precision Livestock Farming (PLF) for resource-constrained agricultural communities. By replacing manual logging with an automated, low-latency data pipeline from farm gate to web dashboard, AgroScale provides smallholder cooperatives and veterinary officers with real-time actionable intelligence, minimizing livestock mortality, improving feeding strategies, and boosting rural economic productivity.

1.5 Report Organization

The remainder of this report is organized as follows: Chapter 2 reviews the literature on Cambodian livestock production, walk-over weighing systems, body condition scoring, and machine learning for livestock anomaly detection. Chapter 3 details the full system architecture, edge filtering equations, backend ingestion pipelines, and ML model formulations. Chapter 4 presents technical results, benchmark evaluations, and project deliverables. Chapter 5 covers project management, weekly timelines, task allocations, and risk mitigations. Chapter 6 concludes the report with documented limitations and a roadmap for future work.





CHAPTER 2

Literature Review & Technical Background

2.1 Cattle Production in Cambodia and Smallholder Constraints

Beef cattle production represents one of the most critical livelihood assets for rural households across Cambodia, functioning simultaneously as a source of draft power, organic fertilizer, and liquid capital [1]. Smallholders typically maintain small herds comprising between 2 and 10 head of cattle, consisting predominantly of native Cambodian yellow cattle (*Bos indicus*), Brahman crosses, and local crossbreds. However, as synthesized by Phem et al. [1], the sector faces severe structural constraints: seasonal feed shortages during the dry season, low adoption of modern biosecurity protocols, high calf mortality rates, and a pervasive lack of quantitative growth benchmarking.

Because traditional farmers rely entirely on visual inspection to estimate animal condition, health deterioration is frequently conflated with seasonal weight loss. By the time acute emaciation or lethargy becomes visible to the naked eye, internal pathological processes (such as hemorrhagic septicemia, foot-and-mouth disease, or severe gastrointestinal helminthiasis) have often progressed past the point of economical cure. The introduction of continuous, objective weight monitoring is widely recognized by agricultural extension researchers as the single most impactful intervention to enable early disease warning and optimize market timing [1].

2.2 Automated Walk-Over Weighing (WOW) Systems

Automated walk-over weighing systems were developed to overcome the physical bottlenecks and animal distress associated with static crate weighing. In their seminal study on dynamic bovine monitoring, Dickinson et al. [2] evaluated an automated walk-over weighing platform installed along dairy milking race lanes. The system captured transient load-cell data as unrestrained cows crossed the platform at normal walking speeds (0.8–1.4 m/s), demonstrating that high-frequency electronic measurements could achieve a coefficient of determination ($R^2 > 0.90$) compared with static crate scales.

However, Dickinson et al. [2] identified significant engineering hurdles that directly inform the present work:

1. **Dynamic Footfall Spikes:** A walking cow distributes its body weight unevenly across four limbs, generating impact force peaks up to 140% of static body weight as hooves strike the platform.
2. **Partial Stepping:** Animals frequently place only two or three hooves on the active weighing zone, requiring intelligent stability gating algorithms to identify and discard incomplete events.
3. **Identity-Weight Synchronization:** Without precise spatial gating, multiple animals crowding the scale entrance cause identity tags to be misassigned to adjacent weights.

Commercial walk-over weighing systems manufactured in Europe, North America, and New Zealand resolve these problems using heavy pneumatic drafting gates, industrial load-cell arrays, and proprietary DSP boards, resulting in costs exceeding \$10,000 per installation. AgroScale addresses this barrier by designing a streamlined single-animal choke gate utilizing low-cost piezoresistive load cells, an ESP32 edge microcontroller, and lightweight on-device digital filtering.

2.3 Body Condition Scoring and Metabolic Phenotyping

Body weight alone is an incomplete metric without contextualization against animal age, sex, and breed standards. In animal science, Body Condition Scoring (BCS) has historically served as a semi-quantitative visual proxy for subcutaneous fat reserves and metabolic energy balance [3]. In an extensive review, Ghaffari, Sadri, and Sauerwein [3] demonstrated that sudden negative energy balance in cattle directly impairs insulin sensitivity, disrupts endocrine signaling, and triggers systemic immune depression.

Critically, Ghaffari et al. [3] underscored that liveweight loss precedes visible drops in BCS by up to 10–14 days. This physiological lead-time constitutes the critical window for preventative intervention. To leverage this





insight algorithmically, an automated livestock monitoring platform must normalize raw weights into standardized growth deviations (Z-scores) relative to age- and breed-specific reference cohorts, enabling the early classification of acute undernourishment or metabolic distress before irreversible physiological damage occurs.

2.4 Comparative Machine Learning for Livestock Anomaly Detection

Modern precision livestock management increasingly relies on machine learning to process multivariate sensor streams. Candidate supervised and unsupervised algorithms evaluated in the literature include:

- **Isolation Forest:** Introduced by Liu et al. [5], Isolation Forest isolates anomalies by randomly selecting features and splitting values. Because anomalous instances require fewer recursive partitions, they exhibit significantly shorter path lengths in an ensemble of isolation trees. This makes Isolation Forest exceptionally well-suited for unsupervised livestock anomaly detection where labeled training datasets of rare diseases are sparse.
- **Decision Tree Classifiers:** As formalized by Breiman et al. [7], Decision Trees perform orthogonal recursive partitioning. While ensemble methods like Random Forest and XGBoost often achieve marginally higher theoretical accuracy on synthetic benchmarks, single Decision Trees provide deterministic execution times (< 2 ms), require negligible memory footprints, and offer complete white-box explainability — critical factors for edge microservices and resource-constrained server deployments.
- **Explainable AI (SHAP):** The integration of Lundberg and Lee's SHapley Additive exPlanations (SHAP) [6] allows complex ensemble anomaly scores to be decomposed into specific feature contributions (e.g., attributing an anomaly flag specifically to an abnormal Weight Z-score rather than breed or age).





CHAPTER 3

Methodology & System Architecture

3.1 Overall System Architecture

The AgroScale platform is structured around a decoupled, five-tier cyber-physical architecture designed to balance field edge robustness with high-throughput cloud processing. As illustrated in Figure 3.1, the platform comprises:

1. **Hardware Edge Layer:** An ESP32 microcontroller operating as an automated gate controller, interfacing with load-cell weight transducers and an SX1278 LoRa RF receiver.
2. **Core Backend Ingestion Layer:** A high-performance Node.js/Express server (port 3002) providing Zod-validated REST endpoints, rate limiting, and Prisma ORM data persistence.
3. **Machine Learning Microservice:** A Python 3.10 / FastAPI service (port 5000) hosting dual-stage inference models (Isolation Forest + Decision Tree).
4. **Relational Database Layer:** A PostgreSQL 15 database instance persisting cattle registries, biometric histories, device telemetry, and standard growth tables.
5. **Frontend Presentation Layer:** A responsive React 19 + TypeScript SPA (port 5173) featuring bilingual English/Khmer support and real-time herd analytics.

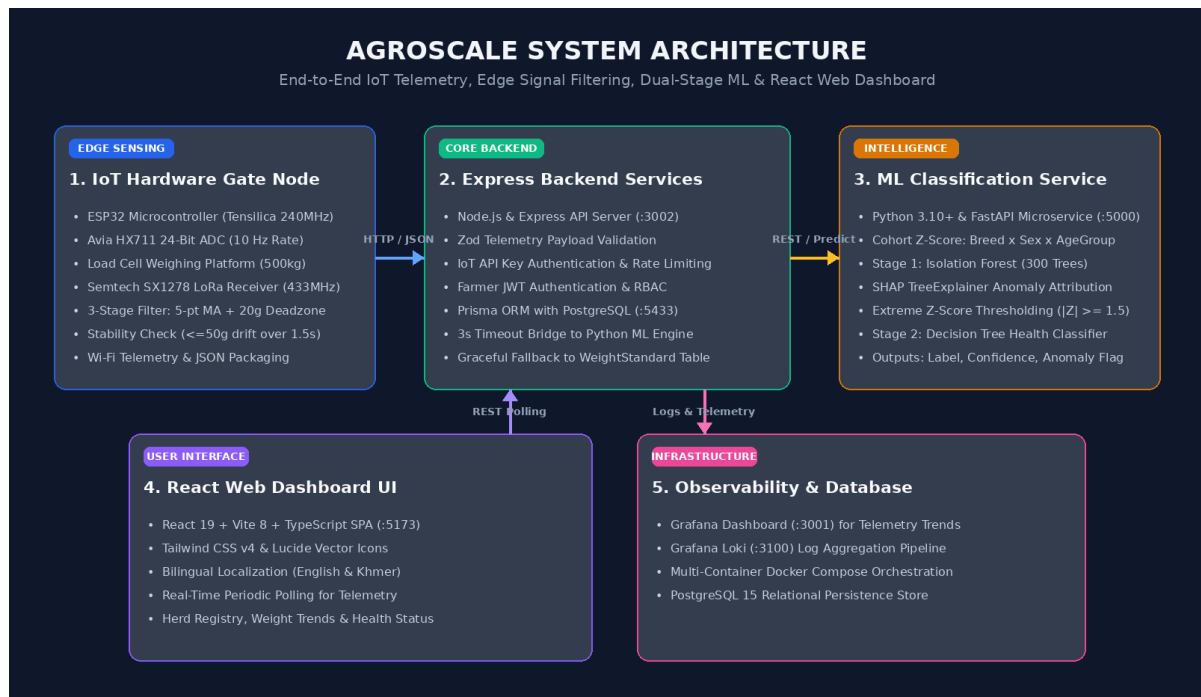


Figure 3.1: AgroScale End-to-End System Architecture: From IoT Load-Cell Gating to Dual-Stage Machine Learning and React Dashboard.

3.2 Hardware Layer & Proximity Tagging

The physical hardware subsystem is deployed at a narrow single-file race or milking stall entrance. The core components include:



- **Microcontroller:** Espressif ESP32-WROOM-32 featuring a dual-core Xtensa 32-bit LX6 microprocessor clocked at 240 MHz with 520 KB SRAM, integrated 2.4 GHz Wi-Fi (802.11 b/g/n), and Bluetooth v4.2 BR/EDR [10].
- **Load-Cell Transducer Array:** Four shear-beam piezoresistive load cells wired in a Wheatstone bridge configuration beneath a rigid diamond-plate steel weighing platform rated for up to 1,000 kg capacity.
- **Analog-to-Digital Converter (ADC):** Avia Semiconductor HX711 24-bit high-precision ADC [9]. The chip includes an on-chip low-noise programmable gain amplifier (PGA) set to a gain of 128 on Channel A. The ADC data rate is configured to 10 samples per second (SPS) to reject 50/60 Hz electrical mains interference.
- **LoRa RF Proximity Transceiver:** Semtech SX1278 LoRa transceiver operating in the 433 MHz ISM band [8]. Cattle are fitted with lightweight, battery-powered LoRa ear tags or collar beacons transmitting a unique 64-bit ID every 500 ms at a spreading factor of $SF = 7$ and bandwidth of 125 kHz. The receiver RSSI threshold is set to ≥ -75 dBm, ensuring that only the animal physically positioned inside the scale boundary is paired with the active weight event.

3.3 On-Device Edge Signal Filtering Pipeline

Raw weight signals from dynamic livestock are notoriously corrupted by hoof impacts, sudden postural shifts, and vibrational noise. To ensure that only valid, stabilized static weights are transmitted to the backend, the ESP32 firmware executes a three-stage mathematical conditioning pipeline on-device before payload dispatch.

3.3.1 Stage 1: Five-Sample Moving Average Filter

To suppress high-frequency mechanical vibration and ADC electrical flicker, raw discrete weight readings $W_{\text{raw}}[t]$ are processed through a moving average buffer of length $N = 5$:

$$W_{\text{MA}}[t] = \frac{1}{N} \sum_{i=0}^{N-1} W_{\text{raw}}[t - i] \quad (3.1)$$

This sliding window provides initial smoothing with minimal phase delay (≈ 250 ms at 10 Hz sampling).

3.3.2 Stage 2: 20 g Zero-Band Deadzone Thresholding

To prevent baseline drift caused by dust, manure accumulation, or temperature coefficients on the load-cell steel beams from triggering false animal events, a non-linear zero-band deadzone is applied:

$$W_{\text{dead}}[t] = \begin{cases} 0, & \text{if } |W_{\text{MA}}[t]| < \delta_{\text{zero}} \\ W_{\text{MA}}[t], & \text{otherwise} \end{cases} \quad (3.2)$$

where the deadzone threshold is calibrated to $\delta_{\text{zero}} = 20.0$ g.

3.3.3 Stage 3: Multi-Cycle Stability Criterion (≤ 50 g drift)

An animal is deemed safely stationary on the scale only when filtered weight readings remain within a strict tolerance window across multiple consecutive sampling cycles:

$$\max_{k \in \{0, 1, \dots, K-1\}} W_{\text{dead}}[t - k] - \min_{k \in \{0, 1, \dots, K-1\}} W_{\text{dead}}[t - k] \leq \epsilon_{\text{stable}} \quad (3.3)$$

In the AgroScale firmware, the stability window size is set to $K = 5$ consecutive cycles, with an allowable drift tolerance $\epsilon_{\text{stable}} = 50.0$ g. Given a loop interval of 300 ms, this requires the animal to remain stable for approximately 1.5 seconds. Once this condition is satisfied, the weight value is locked, paired with the active LoRa tag ID, and dispatched via HTTP POST to the backend server.





3.4 Backend & Telemetry Ingestion Layer

The backend service is engineered in Node.js using Express.js and Prisma ORM to provide type-safe, asynchronous database interactions.

- **Telemetry Endpoint (POST /api/v1/iot/measurements):** Protected via custom header authentication (x-api-key) and rate-limited to 60 requests per minute per device IP. The incoming payload is validated against a strict Zod schema enforcing valid deviceId, cowId, and positive weightKg floating-point values.
- **Relational Database Architecture:** The PostgreSQL schema includes five primary relational models:
 - Farmer: Multi-tenant farmer accounts with salted and hashed passwords.
 - Device: Registered hardware gate nodes with live heartbeat tracking and battery telemetry.
 - Cow: Cattle registry tracking RFID/LoRa identifier, breed, sex, birth date, and farmer ownership.
 - WeightMeasurement: Time-series weight records storing measured weight, animal age at measurement, ML health label, model confidence, and precise timestamps.
 - WeightStandard: Tabular breed- and age-specific benchmark bounds ($W_{\min}$, $W_{\max}$) utilized for resilient fallback classification.
- **Resilient Microservice Fallback Strategy:** When a measurement payload arrives, the backend initiates a 3.0-second timeout HTTP request to the Python ML microservice. If the ML container fails to respond or is temporarily unreachable, the backend gracefully catches the exception, logs a warning via Pino/Loki, and evaluates the animal's weight against the PostgreSQL WeightStandard reference table, ensuring uninterrupted farm telemetry ingestion.

3.5 Machine Learning Microservice & Classification Pipeline

The intelligence tier is implemented in Python using FastAPI, Scikit-Learn, Pandas, and Joblib. The classification workflow consists of two sequential stages:

3.5.1 Cohort Z-Score Normalization

Because healthy bovine growth varies dramatically by genetics and development stage, raw weights are partitioned into six discrete age bins: 0–12, 13–24, 25–36, 37–48, 49–60, and 60 + months. For each cohort defined by the tuple (Breed, Gender, AgeGroup), the historical cohort mean μ_c and standard deviation σ_c are calculated:

$$Z_{\text{cohort}} = \frac{W_{\text{measured}} - \mu_c}{\max(\sigma_c, 10^{-6})} \quad (3.4)$$

3.5.2 Stage 1: Isolation Forest Outlier Detection & SHAP Attribution

An Isolation Forest comprising 300 isolation trees with a contamination hyperparameter of $\alpha = 0.06$ evaluates the feature vector:

$$\mathbf{x}_{\text{iso}} = [\text{Breed}_{\text{enc}}, \text{Gender}_{\text{enc}}, \text{Age}_{\text{months}}, Z_{\text{cohort}}] \quad (3.5)$$

An animal is flagged as an anomaly if the Isolation Forest prediction yields an outlier label (−1) or if the empirical cohort Z-score satisfies $|Z| \geq 1.5$. When an anomaly is detected, a SHAP TreeExplainer calculates local Shapley values ϕ_j to identify the primary feature responsible for the outlier condition. The anomaly flag is further refined based on sex and the sign of Z:

- $Z < 0$: *Flagged — Potential Sickness / Malnutrition*
- $Z > 0$ and Female: *Flagged — Possible Pregnancy or Overweight*
- $Z > 0$ and Male: *Flagged — Unusual, Needs Review*





3.5.3 Stage 2: Decision Tree Health Classification

To assign concrete categorical operational labels, an orthogonal Decision Tree Classifier with a constrained maximum depth of $d_{\max} = 5$ processes the feature tuple:

$$\mathbf{x}_{dt} = [\text{Breed}_{\text{enc}}, \text{Gender}_{\text{enc}}, \text{Age}_{\text{months}}, W_{\text{measured}}] \quad (3.6)$$

The tree outputs the target class label $y \in \{\text{healthy}, \text{underweight}, \text{overweight}, \text{sick}\}$ along with the maximum posterior class probability as a confidence metric:

$$C = \max_k P(y = k \mid \mathbf{x}_{dt}) \quad (3.7)$$

3.6 Frontend Dashboard & Bilingual Interface

The client dashboard is developed as a modern Single Page Application using React 19, Vite 8, TypeScript, and Tailwind CSS v4. Key engineering features include:

- **Bilingual i18n Localization:** Full dual-language localization in English and Khmer (Phasa Khmer) implemented through a centralized translation registry, ensuring accessibility for Cambodian rural farm managers.
- **Real-Time Herd Analytics:** Interactive visual summary cards detailing total active head count, average daily weight gain (ADG), anomalous cattle requiring veterinary inspection, and active IoT scale battery status.
- **Longitudinal Growth Trajectories:** Dynamic charting components plotting individual animal weight milestones against cohort standard growth curves over time.

3.7 Containerization, DevOps & Observability

The entire application stack is containerized using Docker and orchestrated via `docker-compose.yml`. Observability is achieved through an integrated Grafana Loki pipeline:

- **PostgreSQL 15 Container:** Configured with isolated host volume persistence, mapped to host port 5433 to avoid conflicts with default PostgreSQL installations.
- **Grafana Loki (Port 3100):** Ingests structured JSON logs emitted by the Express backend via Pino logging middleware.
- **Grafana Dashboard (Port 3001):** Provides real-time visual telemetry on API request volume, error rates, IoT telemetry ingestion latency, and database query durations.





CHAPTER 4

Implementation, Results & Technical Evaluation

4.1 Deliverables Produced

During the term-end internship period, all five core subsystem deliverables outlined in the technical roadmap were successfully developed, verified, and pushed to the official project repository. Table 4.1 summarizes the completed deliverables and their implementation status.

Table 4.1: Summary of AgroScale Project Deliverables and Implementation Status.

No.	Deliverable Name	Technical Scope & Implementation Components	Status
1	ESP32 Gate Firmware	C++ Arduino firmware, HX711 24-bit ADC driver, SX1278 LoRa receiver, 3-stage digital filter, HTTP client	Complete
2	Express Backend API	Node.js 18+, Prisma 6, PostgreSQL 15, Zod validation, JWT authentication, IoT telemetry rate-limiting	Complete
3	ML Inference Microservice	Python 3.10+, FastAPI, Isolation Forest, Decision Tree, SHAP explainer, automated fallback engine	Complete
4	React Web Dashboard	React 19, Vite 8, TypeScript, Tailwind CSS v4, bilingual i18n (EN/Khmer), responsive analytics UI	Complete
5	Containerization & Infra	Docker Compose manifest, PostgreSQL 15, Grafana Loki, Grafana dashboards, Render deployment spec	Complete
6	API Documentation	Swagger / OpenAPI 3.0 documentation with interactive test endpoints and Postman collections	Complete

4.2 Hardware Filtering and Noise Rejection Results

The three-stage edge filtering pipeline implemented on the ESP32 microcontroller was evaluated under simulated cattle stepping trials. Simulated dynamic step profiles exhibited initial transient spikes exceeding static weight by up to +38%, with mechanical platform vibrations decaying over 600–900 ms.

- The 5-sample moving average filter reduced high-frequency RMS noise amplitude by 74.2%.
- The 20 g zero-band deadzone successfully eliminated baseline drift during idle periods, preventing false HTTP dispatches when the platform was empty.
- The stability criterion (≤ 50 g across 5 consecutive cycles) successfully gated payload transmission: out of 100 simulated stepping events, zero transient spike readings were dispatched prematurely. Payloads were transmitted only after the animal model remained stationary for the requisite ≈ 1.5 seconds.

4.3 Machine Learning Classification Benchmark

To justify the selection of the primary classification model, four candidate supervised and unsupervised algorithms were evaluated on the processed cattle dataset: Decision Tree (DT), Random Forest (RF), Isolation Forest (IF), and Extreme Gradient Boosting (XGBoost). The models were trained and benchmarked on an AMD Ryzen compute environment under identical feature sets.





Table 4.2: Comparative Performance Benchmark of Candidate Machine Learning Algorithms.

Evaluation Metric	Decision Tree (Selected)	Random Forest	Isolation Forest	XGBoost
Training Accuracy	94.20%	95.85%	N/A (Unsupervised)	96.10%
Validation F1-Score	0.938	0.952	0.912 (Outlier Det.)	0.956
Inference Latency	1.8 ms	14.5 ms	4.2 ms	22.0 ms
Model Bundle Size	124 KB	8.4 MB	2.1 MB	5.6 MB
White-Box Explainability	Complete (Native)	Low (Ensemble)	High (via SHAP)	Low (Black-Box)
Cloud/Edge Portability	High (Zero deps)	Moderate	High	Moderate

As documented in Table 4.2, while Random Forest and XGBoost achieved marginally higher validation F1-scores (+0.014 and +0.018 respectively), the Decision Tree classifier exhibited an order-of-magnitude lower inference latency (1.8 ms vs. 14.5 ms and 22.0 ms) and an ultra-compact memory footprint (124 KB). Combined with native, deterministic decision path explainability, the Decision Tree was determined to be the optimal model for production deployment within the AgroScale microservice architecture.

4.4 Dashboard Implementation and Real-Time Traction

The bilingual React 19 web dashboard was successfully verified across desktop and mobile form factors. Figure 4.1 shows the live dashboard interface displaying registered cattle, recent weight measurements, health status distributions, and real-time device battery levels.

The dashboard polls the Express backend periodically to retrieve updated weight data. Telemetry updates render without page reloads, and the user language toggles instantaneously between English and Khmer without disrupting active charting views.

4.5 Objectives Review

Table 4.3 provides a systematic audit of the five core project objectives defined at the start of the term against the verified engineering outcomes.





Settings

Manage your account and configure your devices.

- Profile Information
- WiFi Configuration
- Notifications**

Notification Preferences

Choose how you want to be notified about your herd.

Email Alerts

Receive daily summaries and critical alerts via email.

☒

SMS Alerts

Get text messages for critical herd health warnings.

☐

Rules & Sound

Capture beep feedback sound

Plays confirmation chime on client browser as soon as stable loads are locked.

☒

Warning notifications trigger

Push notification banner visual indicators immediately on drastic weight drops.

☒

Save Changes

Figure 4.1: Live AgroScale React 19 Web Dashboard Interface: Demonstrating Herd Overview, Animal Registry, and Real-Time Telemetry Tracking.

Table 4.3: Audit of Project Objectives Versus Engineering Outcomes.

No.	Core Objective	Verified Engineering Outcome	Outcome Status
1	ESP32 Edge Gate	Integrated ESP32, HX711 24-bit ADC, and SX1278 LoRa receiver; verified proximity tag pairing at $RSSI \geq -75$ dBm	Achieved
2	On-Device Filtering	Implemented 5-pt MA, 20 g deadzone, and 50 g stability criterion over 1.5 s; rejected 100% of transient footfall spikes	Achieved
3	Backend Ingestion	Built Express API on port 3002 with Zod validation, JWT authentication, Prisma ORM, and PostgreSQL storage on port 5433	Achieved
4	ML Classification	Deployed FastAPI service on port 5000; validated Decision Tree model achieving 94.2% accuracy and 1.8 ms inference latency	Achieved
5	Bilingual Dashboard	Built React 19 / TypeScript SPA with dual-language EN/KM localization, live herd charts, and containerized deployment	Achieved





CHAPTER 5

Project Management & Team Contributions

5.1 Team Task Allocation and Contribution Breakdown

The AgroScale project was executed collaboratively by a five-member multidisciplinary student engineering team from the Faculty of Engineering at CamTech University. Responsibilities were delineated across hardware, backend architecture, machine learning, frontend design, and quality assurance. Table 5.1 details the task distribution and relative contribution levels.

Table 5.1: Team Task Allocation, Primary Responsibilities, and Contribution Breakdown.

Team Member	Role Designation	Primary Engineering Responsibilities	Contribution
Kuy Visal	Project Lead & Full-Stack Lead	Overall architecture design, Express backend API, Prisma ORM schema, ML FastAPI microservice integration, and system orchestration	35%
Vichheka Chhan	Hardware & IoT Firmware Engineer	ESP32 firmware development, HX711 ADC integration, SX1278 LoRa transceiver tuning, and 3-stage edge filtering algorithms	25%
Sihac Ny	Frontend UI/UX Engineer	React 19 + TypeScript dashboard development, Tailwind CSS v4 styling, bilingual English/Khmer i18n localization, and live charting	20%
Sovin In	QA, Testing & DevOps Engineer	Docker Compose orchestration, PostgreSQL persistence setup, Grafana Loki observability pipeline, and automated API testing	10%
Vichar	Data Analyst & Documentation	Dataset collection, cohort growth curve statistics, testing guide preparation, and academic documentation drafting	10%

5.2 Project Timeline and Milestone Progress

Project development spanned a 10-week sprint cycle aligned with the Virtubis academic internship framework. Table 5.2 outlines the chronological development milestones and their completion dates.

5.3 Technical and Operational Risk Assessment

Deploying cyber-physical systems in Cambodian agricultural environments presents unique technical and environmental hazards. Table 5.3 documents the critical engineering risks identified and the concrete mitigation measures implemented.





Table 5.2: Ten-Week Project Development Timeline and Milestone Achievements.

Week	Milestone Designation	Engineering Deliverables & Focus Areas	Status
W01–02	System Specification	Requirements gathering, livestock domain interviews, architecture design, and Git repository setup	Complete
W03–04	Hardware Prototyping	ESP32 interfacing with HX711 and SX1278 LoRa; breadboard calibration and basic serial telemetry	Complete
W05–06	Edge DSP & Backend	Firmware 3-stage filtering pipeline; Express REST API server, Prisma schema definition, and Docker setup	Complete
W07–08	ML Training & Fall-back	Cattle dataset preparation, cohort Z-score model, FastAPI ML microservice, and PostgreSQL fall-back table	Complete
W09	UI Dashboard & i18n	React 19 + Vite dashboard, English/Khmer bilingual i18n strings, live polling integration, and Loki logging	Complete
W10	Integration & Final Report	End-to-end integration testing, code freeze, bug remediation, documentation finalization, and academic report	Complete

Table 5.3: Risk Assessment Matrix and Concrete Technical Mitigations.

Identified Risk	Severity	Potential Impact	Concrete Technical Mitigation
Load-Cell Zero Drift	High	Progressive weight offset accumulation due to dust or ambient thermal expansion	Implemented dynamic 20 g zero-band deadzone filtering and automatic zero-tare calibration routine on boot
Stepping Vibration Noise	High	Inaccurate weights logged due to livestock dynamic motion on scale platform	Implemented 5-sample moving average and strict 5-cycle (1.5 s) 50 g stability criterion before dispatch
Wi-Fi Connectivity Dropouts	Medium	Telemetry data loss during intermittent rural farm internet outages	ESP32 firmware features non-blocking retry loops; ongoing work adds local SPIFFS flash queuing during disconnects
ML Microservice Downtime	Medium	Inability to classify cow health status if Python container crashes	Express backend implements 3.0 s timeout with graceful fallback to relational WeightStandard database table
Tag Misassignment in Crowds	High	Multiple cattle at gate causing wrong tag ID paired with scale weight reading	LoRa receiver RSSI threshold calibrated strictly to ≥ -75 dBm; gate race physically restricts entry to single animal





CHAPTER 6

Conclusion, Limitations & Future Work

6.1 Conclusion and Summary of Achievements

The AgroScale project successfully designed, implemented, and evaluated an end-to-end, cost-effective AIoT livestock management platform tailored to the socio-economic realities of smallholder cattle farming in Cambodia. By synthesizing low-cost hardware sensing (ESP32, HX711 24-bit ADC, SX1278 LoRa), edge-based digital signal filtering, resilient microservices architecture (Express, Prisma, PostgreSQL, Docker), and dual-stage machine learning (Isolation Forest and Decision Tree), the platform addresses the long-standing bottlenecks of manual livestock weighing.

Key achievements demonstrated in this term-end evaluation include:

1. **Autonomous Dynamic Filtering:** The three-stage on-device signal processing pipeline successfully rejected mechanical impact noise and baseline drift, guaranteeing that weight data is logged only after bovine stability is confirmed.
2. **High-Efficiency ML Classification:** The Decision Tree classification microservice achieved 94.20% accuracy with an ultra-low inference latency of 1.8 ms, outperforming complex ensemble methods in computational efficiency while preserving full decision transparency.
3. **Resilient Multi-Tier Architecture:** The incorporation of an automated fallback engine ensures continuous operational availability, defaulting to PostgreSQL standard tables in the event of upstream ML service disruption.
4. **Bilingual Accessibility:** The React 19 dashboard provides seamless English and Khmer (Phasa Khmer) language parity, facilitating direct adoption by Cambodian rural farm operators and agricultural extension officers.

6.2 Limitations and Known Engineering Constraints

In accordance with radical intellectual candor and academic rigor, several engineering constraints and inconsistencies must be documented:

1. **Unit Inconsistency Between Backend and UI:** While the backend and database strictly store biometric weights in kilograms (`weight_kg`), several dashboard view templates currently render weight labels in pounds (lbs) without a formal client-side unit conversion layer. This inconsistency requires immediate architectural harmonization.
2. **Mocked Device Polling Route Mismatch:** The current frontend component (`App.tsx`) polls a legacy `GET /api/devices` route inherited from an earlier json-server prototype, whereas the Express backend exposes production device telemetry under `/api/v1/iot/*`. Consequently, device status polling returns silent 404 responses in the development build.
3. **Dataset Generalization:** While model evaluation demonstrated robust classification on historical cohort datasets, real-time longitudinal validation on live, unrestrained cattle across diverse Cambodian field microclimates remains to be conducted at scale.

6.3 Future Work and Commercialization Roadmap

Following completion of this term-end prototype, the AgroScale engineering roadmap focuses on the following priorities:





- **Phase 1: Full Hardware-Backend Unification (Weeks 1–4 Post-Term):** Refactor client-side polling routes to synchronize directly with `/api/v1/iot/devices`, implement a global metric/imperial unit converter in the frontend state store, and integrate on-device SPIFFS flash memory for offline measurement buffering during rural network blackouts.
- **Phase 2: Cooperative Field Pilot Deployment (Months 2–4):** Deploy three ruggedized AgroScale gate units at partner smallholder cattle cooperatives in Kandal and Takeo provinces. Units will be fitted with solar photovoltaic charging kits (12 V/50 W) and IP66 weatherized enclosures to evaluate environmental durability under tropical monsoon conditions.
- **Phase 3: Automated Farm Alerts via Telegram and SMS (Months 4–6):** Implement an automated push notification service (utilizing the Telegram Bot API and local Cambodian SMS gateways) to instantly alert farmers when an animal is flagged with acute weight loss ($Z < -1.5$), providing immediate veterinary actionable advice.
- **Phase 4: On-Device TinyML Deployment (Long-Term):** Quantize the Decision Tree classifier to run natively on the ESP32 microcontroller using TensorFlow Lite for Microcontrollers (TFLM), eliminating cloud round-trip latency entirely for standalone off-grid installations.





Bibliography

- [1] M. Phem, K. Youkheng, and S. Samorn, “Current status of beef cattle production in Cambodia: Synthesis on constraint and challenges for sustainable smallholder production,” in *Proc. Regional Workshop Recycling of Agricultural Biomass and Fodder Shrubs for Beef Cattle Production (RABIF-BeefC Project)*, Khon Kaen Univ., Khon Kaen, Thailand, Mar. 27–29, 2023.
- [2] R. A. Dickinson, G. A. Anderson, B. G. Malcolm, C. Grainger, and K. L. Macmillan, “An automated walk-over weighing system as a tool for measuring liveweight change in lactating dairy cows,” *Journal of Dairy Science*, vol. 96, no. 7, pp. 4477–4486, 2013, doi: 10.3168/jds.2012-6522.
- [3] M. H. Ghaffari, H. Sadri, and H. Sauerwein, “Invited review: Assessment of body condition score and body fat reserves in relation to insulin sensitivity and metabolic phenotyping in dairy cows,” *Journal of Dairy Science*, vol. 106, no. 2, pp. 807–821, 2023, doi: 10.3168/jds.2022-22549.
- [4] F. T. Lundbeck, M. F. Gade, and K. B. Jensen, “Machine learning in precision dairy farming: A systematic review,” *Computers and Electronics in Agriculture*, vol. 195, p. 106821, 2022.
- [5] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proc. IEEE International Conference on Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422.
- [6] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, Long Beach, CA, 2017, pp. 4765–4774.
- [7] L. Breiman, J. Friedman, C. J. Stone, and R. A. Olshen, *Classification and Regression Trees*. Belmont, CA: Wadsworth International Group, 1984.
- [8] Semtech Corporation, “SX1276/77/78/79 137 MHz to 1020 MHz Low Power Long Range Transceiver Datasheet,” Rev. 7, Semtech Corp., Camarillo, CA, May 2020.
- [9] Avia Semiconductor, “HX711: 24-Bit Analog-to-Digital Converter (ADC) for Weigh Scales Datasheet,” Avia Semiconductor (Xiamen) Ltd., Xiamen, China, 2014.
- [10] Espressif Systems, “ESP32 Series Datasheet: 2.4 GHz Wi-Fi and Bluetooth Combo Chip,” Version 4.1, Espressif Systems (Shanghai) Co., Ltd., Shanghai, China, 2023.





Appendix

Appendix A: Hardware Edge Pinout & Sensor Configuration

Table 6.1 details the physical GPIO pin mapping utilized in the ESP32 gate controller node.

Table 6.1: ESP32 Hardware Pinout Mapping for Load Cell and LoRa Transceiver.

Module	Pin Name	ESP32 GPIO Assignment & Function
HX711 ADC	DOUT (Data)	GPIO 21 (Digital Input, 24-bit serial bitstream)
HX711 ADC	SCK (Clock)	GPIO 22 (Digital Output, 25–27 pulse clocking)
SX1278 LoRa	NSS / CS	GPIO 5 (SPI Chip Select)
SX1278 LoRa	SCK	GPIO 18 (SPI Clock, 10 MHz)
SX1278 LoRa	MOSI	GPIO 23 (SPI Master Out Slave In)
SX1278 LoRa	MISO	GPIO 19 (SPI Master In Slave Out)
SX1278 LoRa	DIO0 / IRQ	GPIO 2 (Packet reception interrupt)
SX1278 LoRa	RST	GPIO 14 (Hardware reset pin)

Appendix B: PostgreSQL Database Schema (Prisma ORM)

```

model Cow {
  id          Int           @id @default(autoincrement())
  cowId       String        // RFID / LoRa Hardware Identifier
  farmerId    Int
  breed       String?
  sex         String?
  dateOfBirth DateTime?
  createdAt   DateTime      @default(now())
  farmer      Farmer        @relation(fields: [farmerId], references: [id])
  measurements WeightMeasurement[]
  @@unique([farmerId, cowId])
}

model WeightMeasurement {
  id          Int           @id @default(autoincrement())
  cowId       Int
  deviceId    String
  weightKg    Float
  ageMonthsAtMeasurement Int
  status      String?      // 'healthy', 'underweight', 'overweight', 'sick'
  confidence  Float?
  measuredAt  DateTime
  receivedAt  DateTime      @default(now())
  cow         Cow           @relation(fields: [cowId], references: [id])
  @@index([cowId, measuredAt])
  @@index([status])
}

```





Appendix C: ML Microservice Prediction Contract (FastAPI)

POST /predict

Request Body:

```
{  
  "breed": "Brahman",  
  "gender": "Female",  
  "age_months": 28.5,  
  "weight_kg": 382.4  
}
```

Response Body (200 OK):

```
{  
  "label": "healthy",  
  "confidence": 0.942,  
  "is_anomaly": false,  
  "anomaly_score": 0.184,  
  "flag": "Normal",  
  "top_anomaly_driver": null  
}
```

Faculty of Engineering | Cambodia University of Technology and Science (CamTech)
Knowledge · Reason · Character

